

Discharge Pathway Coordination Service

Cloud Architecture Design Document

Gonzalo Roberto Guerrero Pichén

Cloud Computing in AI

May 2026

Summary: I'm building a GCP-based pathway tracker for a multi-site healthcare provider called **SanaSanaColitaDeRana Corp.** The goal is to match patients to the right discharge routes. It streams in EHR events and structured assessment data, uses a mix of business rules and ML to rank the best pathways, and outputs recommendations to two places: a clinician web dashboard and a consent-gated mobile app for families. Because it's healthcare, we're targeting 99.9% availability, strict regional data residency, and an audit trail for every single recommendation made.

1 Introduction and Requirements

1.1 The problem

When a patient is admitted to a hospital bed, where they go next becomes operationally important within days. The right discharge pathway is rarely just home: it can mean home with a support package, a rehabilitation unit, a community hospital, residential care, or end-of-life care. Each option has different lead times, clinical eligibility, and family preferences. Coordinating across them is largely manual today and is one of the largest sources of delayed transfers of care [1].

Clinicians spend a disproportionate share of their day chasing assessments and bed availability rather than treating patients. Families are often left without a reliable picture of what is supposed to happen next, which contributes to complaints and adverse safety events at the point of transition. This document describes a cloud platform that addresses this gap for SanaSanaColitaDeRana Corp, a fictional multi-site healthcare provider developing a discharge pathway coordination service across its critical care facilities and community sites.

1.2 Domain concepts

Patient flow is the movement of patients through admission, treatment, post-acute care, and discharge. Best practice is for discharge planning to start on admission [1]. A discharge pathway is a packaged combination of clinical setting, support services, and follow-up; common pathways include home with no support, home with a care package, community hospital, specialist rehabilitation, residential or nursing care, and end-of-life care.

Matching a patient to a pathway depends on clinical factors (illness severity, activities of daily living, cognition, medication) and non-clinical factors (housing, family support, transport, finances). The decision sits with the multidisciplinary team; the service does not replace them but surfaces a ranked recommendation with reasons, supports deliberation, and records the resulting decision. This makes the platform a clinical decision support system [2], which means the architecture must be auditable, explainable, and integrated with clinician workflow.

1.3 Goals and non-goals

In scope.

- Track patient state through admission, treatment, and discharge phases in near real-time.

- Ingest patient state events from the hospital EHR and structured assessment results from clinical tools.
- Produce a ranked recommendation of suitable discharge pathways with explanations.
- Provide a clinician-facing dashboard showing patient state, current recommendation, and decision history.
- Provide a family-facing portal with current status and next steps, gated on patient or proxy consent.
- Notify relevant parties when patient state or the recommendation changes.
- Generate handover documents for receiving services.
- Maintain an immutable audit trail of every recommendation and decision.

Out of scope.

- Replacing clinical decision-making. The service produces recommendations; the clinical team decides.
- Real-time vital sign monitoring. Vital signs are consumed from the EHR; this service does not collect them.
- Direct booking of community placements. Bookings continue through the existing booking systems.

1.4 Requirements

ID	Functional requirement
F1	Ingest patient state events from the EHR via a FHIR R5 subscription [4]
F2	Ingest structured assessment results (mobility, cognitive, social)
F3	Remove identifying details (names, dates, locations) from free-text comments before they reach the model
F4	Produce ranked discharge pathway recommendations with reasons
F5	Allow clinicians to view, accept, override, or modify a recommendation
F6	Provide a family facing view of patient state and next steps, gated on consent
F7	Generate handover documents for receiving services
F8	Audit every recommendation and decision immutably

ID	Non-functional requirement
N1	Patient state visible to clinicians in real time
N2	Recommendation generation completes interactively (seconds)
N3	High availability with multi-region failover
N4	Region-bound data residency: data does not leave the original jurisdiction
N5	Compliance with applicable healthcare data protection and clinical decision support standards
N6	Sustained ingest throughput with capacity to absorb peaks
N7	Long term immutable audit retention

1.5 Alternatives considered

Two alternatives were considered for the platform's compute and deployment shape before settling on the current design.

Serverless vs Kubernetes. While a fully serverless stack using Cloud Run or Cloud Functions with Firestore would minimize operational overhead and scale down efficiently outside business hours, it was rejected. The primary drawback was the latency introduced by per request cold starts, which is unacceptable during active clinician ward rounds. Utilizing GKE also natively supports the sidecar pattern required for cross-cutting concerns.

Cloud vs hospital on-premise. A hospital on-premise deployment would give the strongest data residency guarantee but at the cost of significantly slower provisioning, higher fixed infrastructure

cost, and an operational burden that a small platform team cannot realistically carry. A hybrid (data on-premises, compute in cloud) was considered but adds network complexity without improving the security posture meaningfully when the cloud provider already offers private networking within the residency boundary. We opted for a middle-ground approach: a cloud-only deployment restricted to a VPC-bound network.

2 Architecture

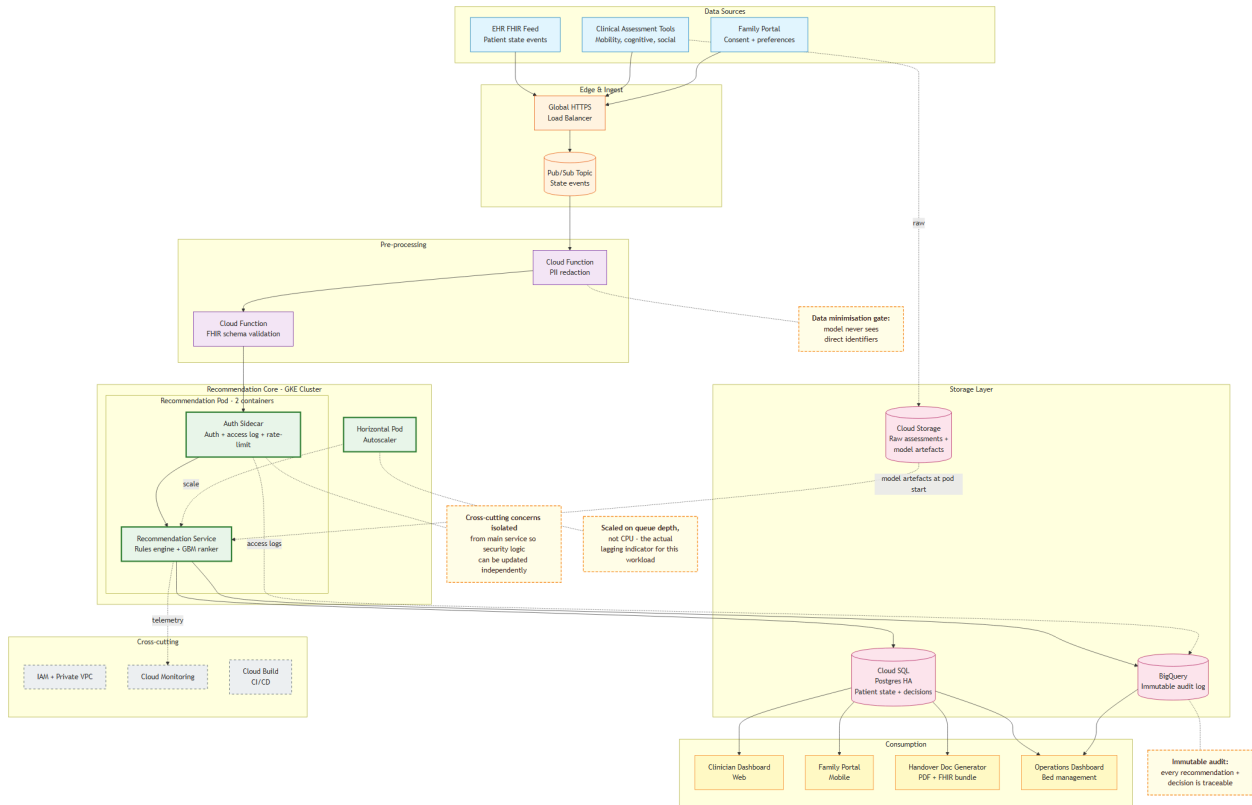


Figure 1: Five horizontal layers (sources, edge, preprocessing, recommendation core, storage); consumption surfaces (clinician dashboard, family portal, handover docs, operations dashboard) above; cross-cutting concerns alongside.

Figure 1 shows the platform as five horizontal layers. Three inputs feed the system: the hospital EHR publishes patient state events over a FHIR R5 subscription [4], clinical assessment tools push structured results, and a family portal records preferences and consent. A Global HTTPS Load Balancer routes traffic into a GKE ingress; the service authenticates callers and publishes events to a Pub/Sub topic. A small Cloud Function applies a PII redaction step to any free-text fields before they reach the model. The recommendation service runs as a Deployment on GKE. Each Pod contains two containers: the recommendation service itself (combining a deterministic rules layer with a gradient-boosted ranking model [3] loaded in-process), and a small sidecar that handles per-clinician auth validation, structured access logging, and basic rate-limiting. Versioned model artefacts are read from a Cloud Storage bucket at pod start-up, and per-pathway feature contributions are computed using SHAP values [9] so that the ranking can be explained to clinicians in a consistent format. Horizontal Pod Autoscaler scales pods on Pub/Sub queue depth. Cloud SQL Postgres (HA, zone-redundant) holds patient state; every recommendation and decision is appended to a BigQuery audit table. The clinician dashboard, family portal, handover document generator, and operations dashboard all read from Cloud SQL. Cross-cutting: IAM with private VPC networking, Cloud Monitoring, and a Cloud Build pipeline applying Terraform and Kubernetes manifests from Git.

2.1 Request walkthrough

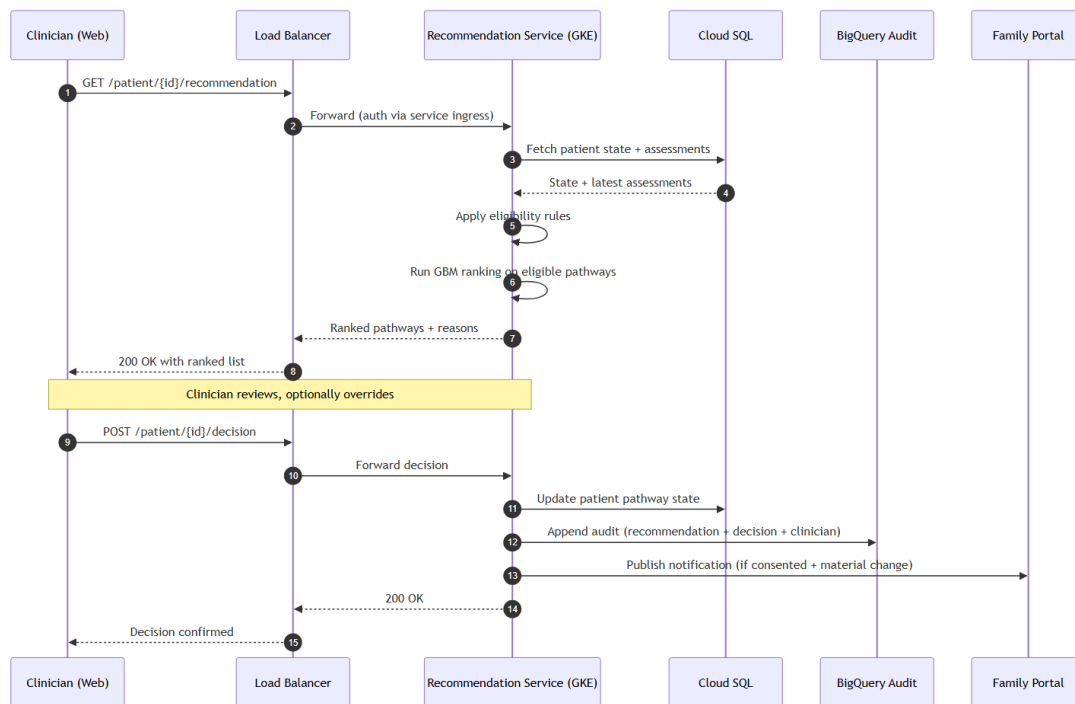


Figure 2: End to end flow for one decision: clinician opens patient view, recommendation is produced, clinician confirms or overrides, state and audit are updated, family is notified.

Figure 2 traces the workflow of an individual decision path. The dashboard queries the recommendation service; the service fetches patient state from Cloud SQL, applies the rules layer, runs the ranking model on the remaining candidates, and returns a ranked list with reasons. The clinician reviews, optionally overrides, and confirms. The decision updates Cloud SQL and is appended to BigQuery; if the family has consented and the change is material, the family portal is notified.

3 Architectural Discussion

This section applies the Well-Architected Framework [5] to the architecture above. For each pillar, the discussion names the design choices that matter most and the trade-offs accepted in exchange.

3.1 Security

Clinical and assessment data is the highest data classification the platform handles, and the architecture treats it that way. Network isolation is achieved by deploying the GKE cluster in a private VPC with no public endpoints other than the Global Load Balancer; data minimisation is enforced by a Cloud Function redaction step that strips direct identifiers from free-text assessment fields before any model inference; immutable audit is provided through a BigQuery table that records every recommendation and decision. Per-clinician controls (auth token validation, access logging, rate-limiting) sit in the sidecar container of the recommendation Pod rather than inside the main service, which keeps the recommendation logic focused on its job and lets the security logic be updated independently. The family portal adds a second authorisation dimension: family access is gated on consent recorded by the patient or by a recognised proxy, and the consent state is itself auditable.

The trade-off is between showing families more in the portal and being careful about who is allowed to see what. Families are not always simple: estranged relatives, disputed proxies, and safeguarding cases all come up. We default to showing less rather than more, knowing that some families will need to ask the ward team for context they cannot get through the portal alone.

3.2 Reliability

When the platform is down, ward-round decisions go back to spreadsheets. Reliability is built in three layers: the service runs across multiple zones in the primary region, a warm standby cluster waits in a secondary region within the same residency boundary, and Pub/Sub holds inbound EHR events for up to seven days. Cloud SQL replicates synchronously across zones and streams to the standby; BigQuery copies the audit table across regions on a schedule; the Global Load Balancer flips traffic to the standby if health checks fail. The trade-off is cost: the standby adds roughly thirty per cent to baseline infrastructure for capacity that sits idle most of the time. Active-active would cut that cost but means coordinating writes across two regions, which is hard to keep safe for clinical state. Active-passive is the right balance for a 99.9 per cent target.

3.3 Performance efficiency

Latency here is less tight than for a real time inference service. The rules-plus-GBM combination meets the three second per recommendation (N2) comfortably on small node sizes, and freshness (N1) is met by pushing EHR updates through Pub/Sub rather than polling for them. Throughput is modest (around 50 events per second sustained), so HPA scales pods on Pub/Sub queue depth, between a floor of two pods and a budget-bound ceiling. GKE was chosen over a serverless option (Cloud Run) because the always-on patient state path benefits from warm capacity rather than scale-from-zero.

3.4 Cost optimisation

The GKE node pool runs at a small steady-state size (a handful of nodes) and lets HPA scale pods within that envelope; the cluster autoscaler adds nodes only at the peak end of the range. Cloud Storage lifecycle moves audit data to coldline at ninety days and archive at one year. Per-site cost is tracked through resource labels so that one site cannot silently absorb the platform's spend. The trade-off is slower retrieval for older audit records, which is acceptable for the use case; the clinician facing path stays on hot infrastructure at all times.

3.5 Operational excellence

Infrastructure is defined in Terraform and services are deployed through a Cloud Build pipeline triggered from Git. Releases roll out to staging, a regression test suite runs over synthetic patient cases, and a clinical lead approves any change to recommendation logic before it reaches production. Each released model version is accompanied by a model card [10] describing intended use, training data composition, performance broken down by demographic group, and known limitations. Cloud Monitoring covers infrastructure and application telemetry with alerts on error rate, latency, and Pub/Sub queue depth. The trade-off is release cadence: model retraining cannot ship as fast as the technical pipeline would allow, but for a clinical decision support tool that constraint seems appropriate.

At larger scale, organisations centralise cross-cutting concerns in a dedicated fleet of edge proxies (Envoy with a control plane pushing configuration at runtime; Syrakis [11] describes an Atlassian-scale implementation with around two thousand proxies across thirteen AWS regions). For SanaSanaColitaDeRana's scale, an edge proxy fleet would be over engineered: one recommendation service does not justify a control plane. The underlying principle, however (centralising rather than duplicating cross-cutting concerns), is what informs the sidecar choice in the recommendation Pod.

3.6 Sustainability

Sustainability is treated as a core concern across three areas (environmental, financial, and long term viability), following the research software engineering framing [6, 7]. Environmental impact is reduced by picking GCP regions with the highest renewable-energy share within the residency boundary, using a small GBM rather than a large neural network, and letting HPA scale pods down during quiet periods. Financial sustainability is supported by right-sizing the infrastructure (small

node pool, HPA-driven scaling) and by tracking cost per site so that no one site silently grows the platform's spend. Long-term viability is supported by preferring open-source dependencies, avoiding lock in to GCP-specific features where standard APIs would do, and keeping documentation alongside the code. Equity, which the Digital Humanities Climate Coalition treats as part of sustainable design [8], is addressed through quarterly demographic bias audits and a fairness gate in the release pipeline.

This pillar carries the design's most uncomfortable trade-off: a ranking model trained on past decisions can reproduce past biases, including ones that disadvantage under-represented patient groups. My own view is that the equity concern is binding. I would block any model release where the demographic disparity in recommendations crosses a small calibration threshold, even if the alternative is to keep running a larger, less energy-efficient model. The environmental cost is real but reversible at the scale of one service; a missed pathway for a vulnerable patient is not.

3.7 Risks and open questions

- **EHR integration variance.** The architecture assumes FHIR R5, but some sites still publish only legacy HL7 v2. A translation service is likely needed at the edge.
- **Family portal consent model.** The consent flow is the most under-specified part of the design and needs sustained input from the safeguarding team before launch.
- **Clinician adoption.** The platform only helps if it is used. The change management plan and ward-level champions are not yet in place.

4 Conclusion

The architecture described in this document keeps the discharge pathway coordination service intentionally small. Where a hyperscaler would centralise cross-cutting concerns in an edge proxy fleet [11], the same principle is applied through a sidecar in a single Pod. Where a more ambitious design would chain a managed PII de-identification service, a model registry, a service-controls perimeter, and a multi-tool GitOps pipeline, this version uses a Cloud Function, a Cloud Storage bucket, a private VPC, and a single Cloud Build pipeline.

The design's primary conflicts sit within the sustainability and equity pillar, where environmental, financial, and clinical objectives run counter to one another. This friction is addressed openly through transparent acknowledgment, backed by a formal commitment to publish bias audits with every model release.

In a second version, the family-portal consent UX would be the priority, ahead of any new clinical features. In a healthcare context, the clarity of what families are consenting to and how easily they can engage with it matters at least as much as any clinical capability the platform could offer; if that interaction is poor, the system loses trust regardless of how good the recommendations are.

5 Glossary

Active-passive / active-active: Multi-region failover patterns. Active-passive runs the primary region at full capacity and a standby region in minimum-capacity reserve. Active-active runs both regions serving traffic at the same time.

BigQuery: Google Cloud's serverless data warehouse; used here for the immutable audit log.

CDSS (Clinical Decision Support System): Software that helps clinicians make decisions by providing relevant information or recommendations, while leaving the decision itself with the clinician.

CI/CD: Continuous Integration / Continuous Deployment. The practice of automatically building, testing, and deploying code from a version-controlled repository.

Cloud Build: Google Cloud's managed CI/CD service.

Cloud Functions: Google Cloud's serverless platform for small, event-driven code; used here for PII redaction and schema validation.

Cloud Run: Google Cloud's serverless container platform; mentioned as a rejected alternative to GKE.

Cloud SQL: Google Cloud's managed relational database service (PostgreSQL in this design).

Cloud Storage: Google Cloud's object storage service; used here for raw assessments and model artefacts.

Cross-cutting concerns: Functionality (such as authentication, logging, rate-limiting) that many parts of a system share, rather than being tied to one feature.

EHR (Electronic Health Record): The digital record of a patient's clinical history maintained by a healthcare provider.

FHIR R5: Fast Healthcare Interoperability Resources, Release 5. An international standard for exchanging healthcare data between systems.

GBM (Gradient Boosting Machine): A class of machine-learning models that combines many small decision trees into a strong predictor.

GCP (Google Cloud Platform): The cloud-computing services offered by Google.

GKE (Google Kubernetes Engine): Google Cloud's managed Kubernetes service for running containerised applications.

HA (High Availability): A system configured to keep running through individual component failures, typically by running redundant components across zones.

HL7 v2: Health Level 7 version 2. An older healthcare messaging standard that predates FHIR but is still widely used in legacy hospital systems.

HPA (Horizontal Pod Autoscaler): Kubernetes component that automatically adjusts the number of Pod replicas based on a chosen metric.

IAM (Identity and Access Management): The service that controls which users and workloads can access which resources.

PII (Personally Identifiable Information): Data that can identify a specific person (names, dates of birth, contact details, etc.).

Pod / Deployment: Kubernetes resources. A Pod is the smallest deployable unit (one or more containers sharing networking); a Deployment manages a set of identical Pods and handles rolling updates and scaling.

Pub/Sub: Google Cloud's managed publish-subscribe messaging service, used to decouple components that produce and consume events.

SHAP: A method for explaining individual model predictions by attributing the output to each input feature; used here to show clinicians why each pathway was ranked where it was.

Sidecar pattern: A Kubernetes pattern where a second container runs in the same Pod as the main service to handle cross-cutting concerns (auth, logging, etc.) without changing the main service.

Terraform: An infrastructure-as-code tool from HashiCorp that defines cloud resources in declarative configuration files.

VPC (Virtual Private Cloud): A network isolated from the public internet, within which the platform runs.

6 References

1. S. Shepperd, N. A. Lannin, L. M. Clemson, et al. "Discharge planning from hospital to home," *Cochrane Database of Systematic Reviews*, 2013.
2. R. T. Sutton, D. Pincock, D. C. Baumgart, et al. "An overview of clinical decision support systems: benefits, risks, and strategies for success," *npj Digital Medicine*, vol. 3, no. 17, 2020.

3. T. Chen and C. Guestrin. “XGBoost: a scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
4. HL7 International. *FHIR Release 5*. <http://hl7.org/fhir/R5/>. Accessed May 2026.
5. Google Cloud. *Google Cloud Well-Architected Framework*. <https://cloud.google.com/architecture/framework>. Accessed May 2026.
6. Software Sustainability Institute. *Midterm Review*, 2023. <https://zenodo.org/records/8205595>.
7. The Turing Way Community. *Guide for Project Design*. The Alan Turing Institute, 2024. <https://book.the-turing-way.org/project-design/project-design/>.
8. Digital Humanities Climate Coalition. *DHCC Toolkit*. <https://sas-dhrh.github.io/dhcc-toolkit/>. Accessed May 2026.
9. S. M. Lundberg and S.-I. Lee. “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
10. M. Mitchell, S. Wu, A. Zaldivar, et al. “Model cards for model reporting,” in *Proc. ACM Conf. on Fairness, Accountability, and Transparency (FAT*)*, 2019, pp. 220–229.
11. V. Syrakakis. “I was laid off by Atlassian” [Video]. YouTube, March 2026. <https://www.youtube.com/watch?v=55pTFVoclvE>.